IT314 – Software Engineering

Lab Assignment 8

Home Appliance Repair Booking System

Student Name: Atharv Trivedi
Student ID: 202401230

Part A – Requirement Analysis

1. System scope

The system models the life cycle of an online home-appliance repair booking. A customer submits appliance and issue details, provides a service address, pays a simulated service charge, and waits for the service center to accept or reject the paid request. An accepted booking progresses through technician assignment, dispatch, service start, and completion.

The proposed system is an in-memory command-line state machine. It does not require a GUI, database, or real payment gateway for this lab.

2. States

State	Meaning	Entry event	Exit event(s) / type
`BOOKING_CREATED`	Booking exists but the request has not been submitted.	Object creation	`BOOK_SERVICE` · Temporary
`PAYMENT_PENDING`	Submitted request awaiting payment.	`BOOK_SERVICE`, `RETRY_PAYMENT`	`PAY`, `CANCEL` · Temporary
`PAYMENT_FAILED`	Latest payment attempt failed; booking is unpaid.	`PAY` failure	`RETRY_PAYMENT`, `CANCEL` · Temporary
`CENTER_REVIEW`	Payment succeeded; the center must decide.	`PAY` success	`ACCEPT_BOOKING`, `REJECT_BOOKING`, `CANCEL` · Temporary
`BOOKING_CONFIRMED`	Center accepted the paid request; technician not yet assigned.	`ACCEPT_BOOKING`	`ASSIGN_TECHNICIAN` · Temporary
`BOOKING_REJECTED`	Center rejected the paid request and refund was issued.	`REJECT_BOOKING`	None · Terminal
`TECHNICIAN_ASSIGNED`	Technician allocated but not yet departed.	`ASSIGN_TECHNICIAN`	`DISPATCH_TECHNICIAN` · Temporary
`TECHNICIAN_EN_ROUTE`	Technician is travelling to the service address.	`DISPATCH_TECHNICIAN`	`START_SERVICE` · Temporary
`SERVICE_IN_PROGRESS`	Technician is performing the repair.	`START_SERVICE`	`COMPLETE_SERVICE` · Temporary
`SERVICE_COMPLETED`	Repair completed successfully.	`COMPLETE_SERVICE`	None · Terminal
`CANCELLED`	Customer cancellation accepted before technician assignment.	`CANCEL`	None · Terminal

State operation rules

State group	Permitted operations	Important invalid operations
`BOOKING_CREATED`	Submit booking details with `BOOK_SERVICE`.	Payment, center, technician, service, and cancel events.
`PAYMENT_PENDING`	Pay or cancel.	Center decision and technician/service events.
`PAYMENT_FAILED`	Retry while the attempt limit permits, or cancel.	Accept, reject, assign, dispatch, start, and complete events.
`CENTER_REVIEW`	Accept, reject, or cancel before assignment.	Technician and service events.
`BOOKING_CONFIRMED`	Assign a technician.	Cancel, dispatch, start, and complete events.
`TECHNICIAN_ASSIGNED` / `TECHNICIAN_EN_ROUTE`	Dispatch, then start service in sequence.	Cancellation, payment, reassignment, and completion out of order.
`SERVICE_IN_PROGRESS`	Complete the repair.	Cancellation and all earlier lifecycle events.
Terminal states	No further operations.	Every event, including cancellation.

3. Events

Event	Actor	Meaning
`BOOK_SERVICE`	Customer	Submit appliance, issue, address, and booking details.
`PAY`	Customer	Attempt simulated payment with success or failure result.
`RETRY_PAYMENT`	Customer	Return from `PAYMENT_FAILED` to `PAYMENT_PENDING` for another attempt.
`ACCEPT_BOOKING`	Service center	Accept a successfully paid request.
`REJECT_BOOKING`	Service center	Reject a successfully paid request and refund it.
`ASSIGN_TECHNICIAN`	Service center	Allocate a technician to an accepted booking.
`DISPATCH_TECHNICIAN`	Technician/service center	Technician leaves for the customer address.
`START_SERVICE`	Technician	Technician arrives and begins repair work.
`COMPLETE_SERVICE`	Technician	Repair work is completed.
`CANCEL`	Customer	Request cancellation, subject to the documented timing guard.

4. Guards

- An event is allowed only when the current state has a matching transition.
- Payment must be successful before service-center processing can begin.

- A payment attempt is allowed only while fewer than three total attempts have been made.
- `ACCEPT_BOOKING` and `REJECT_BOOKING` require the `CENTER_REVIEW` state and a successful payment.
- Technician and service events must occur in their specified sequence.
- Cancellation is allowed only before technician assignment: from `PAYMENT_PENDING`, `PAYMENT_FAILED`, or `CENTER_REVIEW`.
- An invalid event never changes the booking state or payment record.

5. Actions

- `BOOK_SERVICE`: store the submitted request and wait for payment.
- Successful `PAY`: mark payment as paid and move to center review.
- Failed `PAY`: increment the attempt counter, keep the payment unpaid, and enter `PAYMENT_FAILED`.
- `RETRY_PAYMENT`: permit one more attempt when the retry limit has not been reached.
- `REJECT_BOOKING`: move to terminal rejection and issue a full simulated refund.
- `CANCEL`: move to terminal cancellation; refund the full charge if it had been paid, otherwise issue no refund.
- Technician/service transitions print the previous state, event, new state, and action.

6. Invalid operations

Examples include:

- `COMPLETE_SERVICE` from `PAYMENT_PENDING`.
- `ACCEPT_BOOKING` before payment succeeds.
- `ASSIGN_TECHNICIAN` after the center rejects a booking.
- `CANCEL` after `TECHNICIAN_ASSIGNED`, `SERVICE_IN_PROGRESS`, or `SERVICE_COMPLETED`.
- Any event after a terminal state.

Each produces an `INVALID TRANSITION` message and explicitly reports that the state is unchanged.

Part B – States and Transitions

1. Transition table

Current state	Event	Guard	Next state	Action	Valid?
`BOOKING_CREATED`	`BOOK_SERVICE`	Request details are present	`PAYMENT_PENDING`	Accept booking details	Yes
`PAYMENT_PENDING`	`PAY(success)`	Attempts < 3	`CENTER_REVIEW`	Mark payment successful	Yes
`PAYMENT_PENDING`	`PAY(failure)`	Attempts < 3	`PAYMENT_FAILED`	Keep booking unpaid	Yes
`PAYMENT_FAILED`	`RETRY_PAYMENT`	Attempts < 3	`PAYMENT_PENDING`	Permit another attempt	Yes
`PAYMENT_FAILED`	`RETRY_PAYMENT`	Attempts = 3	Unchanged	Reject retry	No
`CENTER_REVIEW`	`ACCEPT_BOOKING`	Payment is successful	`BOOKING_CONFIRMED`	Center accepts request	Yes
`CENTER_REVIEW`	`REJECT_BOOKING`	Payment is successful	`BOOKING_REJECTED`	Full simulated refund	Yes
`PAYMENT_PENDING`	`CANCEL`	Before assignment; unpaid	`CANCELLED`	No refund required	Yes
`PAYMENT_FAILED`	`CANCEL`	Before assignment; unpaid	`CANCELLED`	No refund required	Yes
`CENTER_REVIEW`	`CANCEL`	Before assignment; paid	`CANCELLED`	Full simulated refund	Yes
`BOOKING_CONFIRMED`	`ASSIGN_TECHNICIAN`	Center accepted	`TECHNICIAN_ASSIGNED`	Assign technician	Yes
`TECHNICIAN_ASSIGNED`	`DISPATCH_TECHNICIAN`	Technician assigned	`TECHNICIAN_EN_ROUTE`	Dispatch technician	Yes
`TECHNICIAN_EN_ROUTE`	`START_SERVICE`	Technician arrived	`SERVICE_IN_PROGRESS`	Begin repair	Yes
`SERVICE_IN_PROGRESS`	`COMPLETE_SERVICE`	Repair finished	`SERVICE_COMPLETED`	Mark service	Yes

Current state	Event	Guard	Next state	Action	Valid?
				complete	
Any state	Event not listed for that state	No matching transition	Unchanged	Print invalid-transition message	No

2. Successful path

```
BOOKING_CREATED
  --BOOK_SERVICE⟶
PAYMENT_PENDING
  --PAY(success)⟶
CENTER_REVIEW
  --ACCEPT_BOOKING⟶
BOOKING_CONFIRMED
  --ASSIGN_TECHNICIAN⟶
TECHNICIAN_ASSIGNED
  --DISPATCH_TECHNICIAN⟶
TECHNICIAN_EN_ROUTE
  --START_SERVICE⟶
SERVICE_IN_PROGRESS
  --COMPLETE_SERVICE⟶
SERVICE_COMPLETED
```

3. Exceptional paths

```
PAYMENT_PENDING --PAY(failure)⟶ PAYMENT_FAILED
PAYMENT_FAILED --RETRY_PAYMENT [attempts < 3]⟶ PAYMENT_PENDING
CENTER_REVIEW --REJECT_BOOKING / full refund⟶ BOOKING_REJECTED
CENTER_REVIEW --CANCEL / full refund⟶ CANCELLED
```

Part C – Cancellation and Payment Design

1. Payment behavior

Payment is simulated using a boolean success/failure input. No external gateway is used. A successful attempt marks the payment as paid and moves the booking to `CENTER_REVIEW`. A failed attempt increments the attempt counter and moves the booking to `PAYMENT_FAILED`; the booking is not treated as paid.

The maximum number of total payment attempts is **three**. A retry is allowed only from `PAYMENT_FAILED` and only while an attempt remains. The third attempt is the final permitted attempt; a later retry is rejected without changing state.

2. Service-center rejection

A paid booking can be rejected only from `CENTER_REVIEW`. Rejection moves the booking to terminal `BOOKING_REJECTED` and issues a full simulated refund. Because the state is terminal, technician assignment is rejected afterward.

3. Cancellation policy

Cancellation is allowed only before technician assignment:

- From `PAYMENT_PENDING`: cancellation moves to `CANCELLED`; no refund is needed because payment has not succeeded.
- From `PAYMENT_FAILED`: cancellation moves to `CANCELLED`; no refund is needed.
- From `CENTER_REVIEW`: cancellation moves to `CANCELLED`; the successful payment receives a full simulated refund.

Cancellation is rejected from `BOOKING_CONFIRMED`, `TECHNICIAN_ASSIGNED`, `TECHNICIAN_EN_ROUTE`, `SERVICE_IN_PROGRESS`, and `SERVICE_COMPLETED`. The state remains unchanged. This prevents cancellation after the service center has accepted and committed operational resources, and prevents cancellation after work has begun or finished.

4. Boundary condition

The near-boundary condition is the transition between `CENTER_REVIEW` and `BOOKING_CONFIRMED`. Cancellation in `CENTER_REVIEW` is allowed with a full refund; cancellation immediately after acceptance, in `BOOKING_CONFIRMED`, is rejected. The implementation tests both post-service boundaries as well: cancellation is rejected in `SERVICE_IN_PROGRESS` and `SERVICE_COMPLETED`.

The payment boundary is also explicit: three failed attempts are permitted, but a fourth attempt is not. These rules are enforced consistently by the state-machine design.

5. Traceability to implementation

The implementation artifacts consist of:

- A complete state enumeration for the booking lifecycle.
- A deterministic payment-attempt and refund simulation.
- A booking object containing data, event validation, and state transitions.
- An interactive CLI and automated TC01–TC07 driver.

The UML diagram, source code, CLI output, and test-results PDF use the same state names, guards, actions, cancellation rule, and refund behavior described in this report.